

journal.c

Full source code export

Lines: 485

Source path: /mnt/data/journal.c

```

1: #include <errno.h>
2: #include <fcntl.h>
3: #include <stdint.h>
4: #include <stdio.h>
5: #include <stdlib.h>
6: #include <string.h>
7: #include <time.h>
8: #include <unistd.h>
9:
10: #define FS_MAGIC 0x56534653U
11: #define JOURNAL_MAGIC 0x4A524E4CU // "JRNL"
12:
13: #define BLOCK_SIZE      4096U
14: #define INODE_SIZE      128U
15: #define JOURNAL_BLOCK_IDX  1U
16: #define JOURNAL_BLOCKS    16U
17: #define INODE_BLOCKS      2U
18: #define DATA_BLOCKS      64U
19: #define INODE_BMAP_IDX    (JOURNAL_BLOCK_IDX + JOURNAL_BLOCKS)
20: #define DATA_BMAP_IDX    (INODE_BMAP_IDX + 1U)
21: #define INODE_START_IDX   (DATA_BMAP_IDX + 1U)
22: #define DATA_START_IDX   (INODE_START_IDX + INODE_BLOCKS)
23: #define TOTAL_BLOCKS      (DATA_START_IDX + DATA_BLOCKS)
24: #define DEFAULT_IMAGE "vsfs.img"
25:
26: #define REC_DATA  1
27: #define REC_COMMIT 2
28:
29: #define NAME_LEN 28
30:
31: #define INODE_TYPE_FREE 0
32: #define INODE_TYPE_FILE 1
33: #define INODE_TYPE_DIR 2
34:
35: struct superblock {
36:     uint32_t magic;
37:     uint32_t block_size;
38:     uint32_t total_blocks;
39:     uint32_t inode_count;
40:     uint32_t journal_block;
41:     uint32_t inode_bitmap;
42:     uint32_t data_bitmap;
43:     uint32_t inode_start;
44:     uint32_t data_start;
45:     uint8_t _pad[128 - 9 * 4];
46: };
47:
48: struct inode {
49:     uint16_t type;
50:     uint16_t links;
51:     uint32_t size;
52:     uint32_t direct[8];
53:     uint32_t ctime;
54:     uint32_t mtime;
55:     uint8_t _pad[128 - (2 + 2 + 4 + 8 * 4 + 4 + 4)];
56: };
57:
58: struct dirent {
59:     uint32_t inode;
60:     char name[NAME_LEN];
61: };
62:
63: struct journal_header {

```

```

64:     uint32_t magic;
65:     uint32_t nbytes_used;
66: };
67:
68: struct rec_header {
69:     uint16_t type;
70:     uint16_t size;
71: };
72:
73: struct data_record {
74:     struct rec_header hdr;
75:     uint32_t block_no;
76:     uint8_t data[BLOCK_SIZE];
77: };
78:
79: struct commit_record {
80:     struct rec_header hdr;
81: };
82:
83: static void die(const char *msg) {
84:     perror(msg);
85:     exit(EXIT_FAILURE);
86: }
87:
88: static void read_block(int fd, uint32_t block_no, void *buf) {
89:     if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {
90:         die("lseek");
91:     }
92:     if (read(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {
93:         die("read");
94:     }
95: }
96:
97: static void write_block(int fd, uint32_t block_no, const void *buf) {
98:     if (lseek(fd, block_no * BLOCK_SIZE, SEEK_SET) < 0) {
99:         die("lseek");
100:     }
101:     if (write(fd, buf, BLOCK_SIZE) != BLOCK_SIZE) {
102:         die("write");
103:     }
104: }
105:
106: static void read_journal_header(int fd, struct journal_header *jh) {
107:     if (lseek(fd, JOURNAL_BLOCK_IDX * BLOCK_SIZE, SEEK_SET) < 0) {
108:         die("lseek");
109:     }
110:     if (read(fd, jh, sizeof(struct journal_header)) != sizeof(struct journal_header))
111:     {
112:         die("read journal header");
113:     }
114: }
115:
116: static void write_journal_header(int fd, const struct journal_header *jh) {
117:     if (lseek(fd, JOURNAL_BLOCK_IDX * BLOCK_SIZE, SEEK_SET) < 0) {
118:         die("lseek");
119:     }
120:     if (write(fd, jh, sizeof(struct journal_header)) != sizeof(struct journal_header))
121:     {
122:         die("write journal header");
123:     }
124: }
125:
126: static int get_bitmap_bit(const uint8_t *bitmap, uint32_t index) {

```

```

125:     return (bitmap[index / 8] & (1U << (index % 8))) != 0;
126: }
127:
128: static void set_bitmap_bit(uint8_t *bitmap, uint32_t index) {
129:     bitmap[index / 8] |= (1U << (index % 8));
130: }
131:
132: static uint32_t find_free_inode(const uint8_t *inode_bitmap, uint32_t max_inodes) {
133:     for (uint32_t i = 0; i < max_inodes; i++) {
134:         if (!get_bitmap_bit(inode_bitmap, i)) {
135:             return i;
136:         }
137:     }
138:     return (uint32_t)-1;
139: }
140:
141: static int journal_append_data(int fd, struct journal_header *jh, uint32_t block_no,
    const void *block_data) {
142:     uint32_t rec_size = sizeof(struct rec_header) + sizeof(uint32_t) + BLOCK_SIZE;
143:
144:     if (jh->nbytes_used + rec_size > JOURNAL_BLOCKS * BLOCK_SIZE) {
145:         fprintf(stderr, "Journal full! Run install first.\n");
146:         return -1;
147:     }
148:
149:     struct rec_header rec_hdr;
150:     rec_hdr.type = REC_DATA;
151:     rec_hdr.size = (uint16_t)rec_size;
152:
153:     off_t offset = JOURNAL_BLOCK_IDX * BLOCK_SIZE + jh->nbytes_used;
154:     if (lseek(fd, offset, SEEK_SET) < 0) {
155:         die("lseek");
156:     }
157:
158:
159:     if (write(fd, &rec_hdr, sizeof(rec_hdr)) != sizeof(rec_hdr)) {
160:         die("write");
161:     }
162:
163:
164:     if (write(fd, &block_no, sizeof(block_no)) != sizeof(block_no)) {
165:         die("write");
166:     }
167:
168:
169:     if (write(fd, block_data, BLOCK_SIZE) != BLOCK_SIZE) {
170:         die("write");
171:     }
172:
173:     jh->nbytes_used += rec_size;
174:     return 0;
175: }
176:
177: static int journal_append_commit(int fd, struct journal_header *jh) {
178:     uint32_t rec_size = sizeof(struct rec_header);
179:
180:     if (jh->nbytes_used + rec_size > JOURNAL_BLOCKS * BLOCK_SIZE) {
181:         fprintf(stderr, "Journal full! Run install first.\n");
182:         return -1;
183:     }
184:
185:     struct rec_header rec_hdr;
186:     rec_hdr.type = REC_COMMIT;

```

```

187:     rec_hdr.size = (uint16_t)rec_size;
188:
189:     off_t offset = JOURNAL_BLOCK_IDX * BLOCK_SIZE + jh->nbytes_used;
190:     if (lseek(fd, offset, SEEK_SET) < 0) {
191:         die("lseek");
192:     }
193:     if (write(fd, &rec_hdr, rec_size) != (ssize_t)rec_size) {
194:         die("write");
195:     }
196:
197:     jh->nbytes_used += rec_size;
198:
199:     write_journal_header(fd, jh);
200:
201:     return 0;
202: }
203:
204: static void cmd_create(const char *filename) {
205:     int fd = open(DEFAULT_IMAGE, O_RDWR);
206:     if (fd < 0) {
207:         die("open");
208:     }
209:
210:     struct superblock sb;
211:     read_block(fd, 0, &sb);
212:
213:     if (sb.magic != FS_MAGIC) {
214:         fprintf(stderr, "Invalid filesystem magic\n");
215:         close(fd);
216:         exit(EXIT_FAILURE);
217:     }
218:
219:
220:     struct journal_header jh;
221:     read_journal_header(fd, &jh);
222:
223:
224:     if (jh.magic != JOURNAL_MAGIC) {
225:         jh.magic = JOURNAL_MAGIC;
226:         jh.nbytes_used = sizeof(struct journal_header);
227:         write_journal_header(fd, &jh);
228:     }
229:
230:
231:     uint8_t inode_bitmap[BLOCK_SIZE];
232:     uint8_t data_bitmap[BLOCK_SIZE];
233:     read_block(fd, INODE_BMAP_IDX, inode_bitmap);
234:     read_block(fd, DATA_BMAP_IDX, data_bitmap);
235:
236:
237:     uint32_t new_inode_idx = find_free_inode(inode_bitmap, sb.inode_count);
238:     if (new_inode_idx == (uint32_t)-1) {
239:         fprintf(stderr, "No free inodes\n");
240:         close(fd);
241:         exit(EXIT_FAILURE);
242:     }
243:
244:
245:     uint8_t root_inode_block[BLOCK_SIZE];
246:     read_block(fd, INODE_START_IDX, root_inode_block);
247:     struct inode *root_inode = (struct inode *)root_inode_block;
248:
249:     if (root_inode->type != INODE_TYPE_DIR) {

```

```

250:         fprintf(stderr, "Root inode is not a directory\n");
251:         close(fd);
252:         exit(EXIT_FAILURE);
253:     }
254:
255:     uint32_t root_dir_block = root_inode->direct[0];
256:     uint8_t root_data[BLOCK_SIZE];
257:     read_block(fd, root_dir_block, root_data);
258:
259:     struct dirent *dirents = (struct dirent *)root_data;
260:     int free_slot = -1;
261:
262:
263:
264:     for (int i = 2; i < (int)(BLOCK_SIZE / sizeof(struct dirent)); i++) {
265:         if (dirents[i].inode == 0 && free_slot == -1) {
266:             free_slot = i;
267:         }
268:         if (dirents[i].inode != 0 && strcmp(dirents[i].name, filename) == 0) {
269:             fprintf(stderr, "File '%s' already exists\n", filename);
270:             close(fd);
271:             exit(EXIT_FAILURE);
272:         }
273:     }
274:
275:     if (free_slot == -1) {
276:         fprintf(stderr, "Root directory full\n");
277:         close(fd);
278:         exit(EXIT_FAILURE);
279:     }
280:
281:
282:     set_bitmap_bit(inode_bitmap, new_inode_idx);
283:
284:
285:     dirents[free_slot].inode = new_inode_idx;
286:     strncpy(dirents[free_slot].name, filename, NAME_LEN - 1);
287:     dirents[free_slot].name[NAME_LEN - 1] = '\0';
288:
289:
290:     root_inode->size += sizeof(struct dirent);
291:     root_inode->mtime = (uint32_t)time(NULL);
292:
293:
294:     uint32_t inode_block_idx = INODE_START_IDX + (new_inode_idx * INODE_SIZE) /
BLOCK_SIZE;
295:     uint32_t inode_offset = (new_inode_idx * INODE_SIZE) % BLOCK_SIZE;
296:
297:
298:     if (inode_block_idx == INODE_START_IDX) {
299:
300:         struct inode *new_inode = (struct inode *)(root_inode_block + inode_offset);
301:         memset(new_inode, 0, sizeof(struct inode));
302:         new_inode->type = INODE_TYPE_FILE;
303:         new_inode->links = 1;
304:         new_inode->size = 0;
305:         new_inode->ctime = (uint32_t)time(NULL);
306:         new_inode->mtime = (uint32_t)time(NULL);
307:
308:
309:         if (journal_append_data(fd, &jh, INODE_BMAP_IDX, inode_bitmap) < 0) {
310:             close(fd);
311:             exit(EXIT_FAILURE);

```

```

312:     }
313:
314:
315:     if (journal_append_data(fd, &jh, INODE_START_IDX, root_inode_block) < 0) {
316:         close(fd);
317:         exit(EXIT_FAILURE);
318:     }
319: } else {
320:
321:     uint8_t new_inode_block[BLOCK_SIZE];
322:     read_block(fd, inode_block_idx, new_inode_block);
323:
324:     struct inode *new_inode = (struct inode *) (new_inode_block + inode_offset);
325:     memset(new_inode, 0, sizeof(struct inode));
326:     new_inode->type = INODE_TYPE_FILE;
327:     new_inode->links = 1;
328:     new_inode->size = 0;
329:     new_inode->ctime = (uint32_t)time(NULL);
330:     new_inode->mtime = (uint32_t)time(NULL);
331:
332:
333:     if (journal_append_data(fd, &jh, INODE_BMAP_IDX, inode_bitmap) < 0) {
334:         close(fd);
335:         exit(EXIT_FAILURE);
336:     }
337:
338:
339:     if (journal_append_data(fd, &jh, inode_block_idx, new_inode_block) < 0) {
340:         close(fd);
341:         exit(EXIT_FAILURE);
342:     }
343:
344:
345:     if (journal_append_data(fd, &jh, INODE_START_IDX, root_inode_block) < 0) {
346:         close(fd);
347:         exit(EXIT_FAILURE);
348:     }
349: }
350:
351:
352: if (journal_append_data(fd, &jh, root_dir_block, root_data) < 0) {
353:     close(fd);
354:     exit(EXIT_FAILURE);
355: }
356:
357:
358: if (journal_append_commit(fd, &jh) < 0) {
359:     close(fd);
360:     exit(EXIT_FAILURE);
361: }
362:
363: close(fd);
364: printf("Logged creation of '%s' to journal.\n", filename);
365: }
366:
367: static void cmd_install(void) {
368:     int fd = open(DEFAULT_IMAGE, O_RDWR);
369:     if (fd < 0) {
370:         die("open");
371:     }
372:
373:
374:     struct journal_header jh;

```

```

375:     read_journal_header(fd, &jh);
376:
377:     if (jh.magic != JOURNAL_MAGIC) {
378:         printf("Journal is empty or uninitialized.\n");
379:         close(fd);
380:         return;
381:     }
382:
383:
384:     typedef struct {
385:         uint32_t block_no;
386:         uint8_t block_data[BLOCK_SIZE];
387:     } pending_write;
388:
389:     pending_write *writes = malloc(sizeof(pending_write) * 100);
390:     if (!writes) {
391:         die("malloc");
392:     }
393:
394:     uint32_t offset = sizeof(struct journal_header);
395:     int txn_count = 0;
396:     int write_count = 0;
397:
398:     while (offset < jh.nbytes_used) {
399:
400:         struct rec_header rec_hdr;
401:         if (lseek(fd, JOURNAL_BLOCK_IDX * BLOCK_SIZE + offset, SEEK_SET) < 0) {
402:             die("lseek");
403:         }
404:
405:         ssize_t bytes_read = read(fd, &rec_hdr, sizeof(rec_hdr));
406:         if (bytes_read != sizeof(rec_hdr)) {
407:             break;
408:         }
409:
410:         if (rec_hdr.type == REC_COMMIT) {
411:
412:             for (int i = 0; i < write_count; i++) {
413:                 write_block(fd, writes[i].block_no, writes[i].block_data);
414:             }
415:             txn_count++;
416:             write_count = 0;
417:             offset += rec_hdr.size;
418:
419:         } else if (rec_hdr.type == REC_DATA) {
420:
421:             uint32_t block_no;
422:             if (read(fd, &block_no, sizeof(block_no)) != sizeof(block_no)) {
423:                 break;
424:             }
425:
426:
427:             uint8_t block_data[BLOCK_SIZE];
428:             if (read(fd, block_data, BLOCK_SIZE) != BLOCK_SIZE) {
429:                 break;
430:             }
431:
432:
433:             if (write_count < 100) {
434:                 writes[write_count].block_no = block_no;
435:                 memcpy(writes[write_count].block_data, block_data, BLOCK_SIZE);
436:                 write_count++;
437:             }

```

```
438:
439:         offset += rec_hdr.size;
440:
441:     } else {
442:
443:         break;
444:     }
445: }
446:
447:
448: if (write_count > 0) {
449:     printf("Warning: %d uncommitted DATA records discarded.\n", write_count);
450: }
451:
452: free(writes);
453:
454:
455: memset(&jh, 0, sizeof(jh));
456: jh.magic = JOURNAL_MAGIC;
457: jh.nbytes_used = sizeof(struct journal_header);
458: write_journal_header(fd, &jh);
459:
460: close(fd);
461: printf("Installed %d committed transaction%s from journal.\n",
462:        txn_count, txn_count == 1 ? "" : "s");
463: }
464:
465: int main(int argc, char *argv[]) {
466:     if (argc < 2) {
467:         fprintf(stderr, "Usage: %s <create|install> [filename]\n", argv[0]);
468:         return EXIT_FAILURE;
469:     }
470:
471:     if (strcmp(argv[1], "create") == 0) {
472:         if (argc < 3) {
473:             fprintf(stderr, "Usage: %s create <filename>\n", argv[0]);
474:             return EXIT_FAILURE;
475:         }
476:         cmd_create(argv[2]);
477:     } else if (strcmp(argv[1], "install") == 0) {
478:         cmd_install();
479:     } else {
480:         fprintf(stderr, "Unknown command: %s\n", argv[1]);
481:         return EXIT_FAILURE;
482:     }
483:
484:     return EXIT_SUCCESS;
485: }
```